

Mobile Price Classification

Support Vector Machines (SVM)

Presented by: **Cotton, Flor, Kai and Yuna**



Introduction to the dataset and research paper

Flor



The dataset

Mobile Price Classification

Bob has started a mobile phone company and wants to compete with major brands. His main challenge is determining the appropriate price range for the phones his company produces.

To solve this problem, he collects mobile phone sales data and analyzes the relationship between technical features, and the phone's selling price.

The goal is not to predict the exact price, but to classify each mobile phone into a price range category based on its features.

[Dataset](#)



2,000
rows

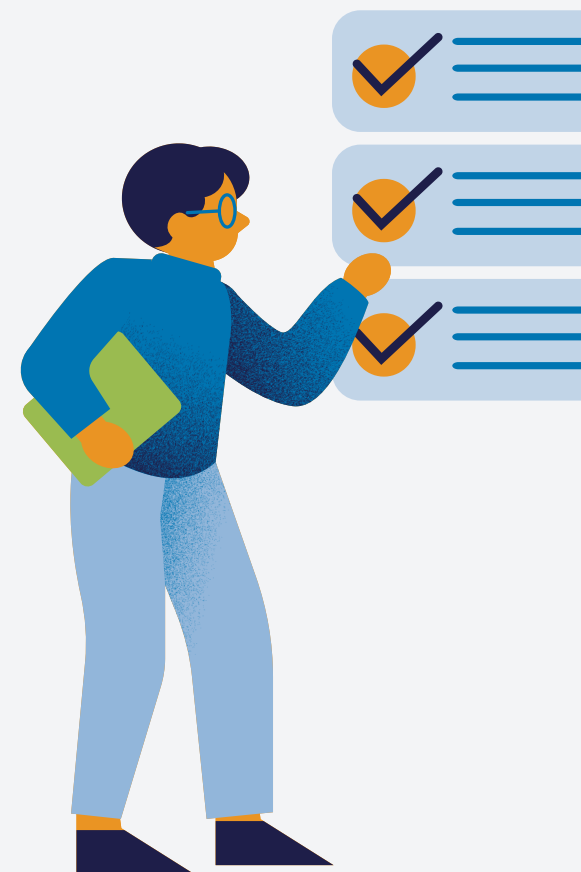
21
variables

0
missing values

500
phones per class

Target distribution

0 Low		500 (25%)
1 Medium		500 (25%)
2 High		500 (25%)
3 Very High		500 (25%)



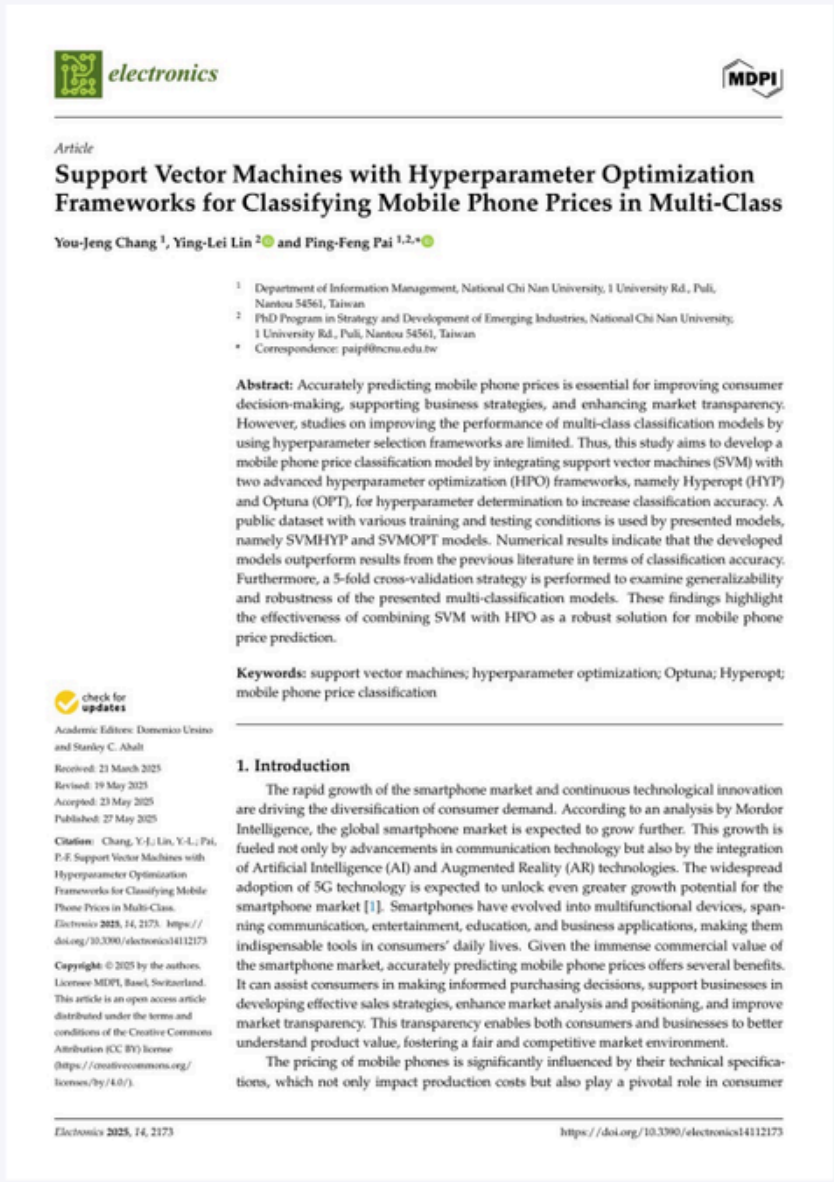
Variable	Meaning
battery_power	The capacity of the phone's battery.
blue	Indicates whether the phone supports Bluetooth. 0 = No 1, = Yes
clock_speed	Clock speed measures how fast the phone's processor operates.
dual_sim	Indicates whether the phone supports Dual. 0 = No 1 = Yes
fc	FC stands for front camera and represents its resolution in megapixels.
four_g	Four-G indicates whether the phone supports 4G connectivity.
int_memory	The amount of storage available on the phone.

Variable	Meaning
m_dep	Mobile depth represents the thickness of the phone.
mobile_wt	The physical weight of the device in grams.
n_cores	How many processing units the phone's processor contains.
pc	The resolution of the primary or rear camera.
px_height	The vertical resolution of the phone's display.
px_width	The horizontal resolution of the display.
ram	The temporary memory used by the phone to run applications

Variable	Meaning
sc_h	Screen height represents the physical height of the phone's screen.
sc_w	Screen width represents the physical width of the screen.
talk_time	Maximum time that the battery can last on a single charge.
three_g	Indicates whether the phone supports 3G connectivity.
touch_screen	Indicates whether the phone supports a touch screen.
wifi	Indicates whether the phone supports wireless internet connectivity.
price_range	Predict which price category it belongs to.

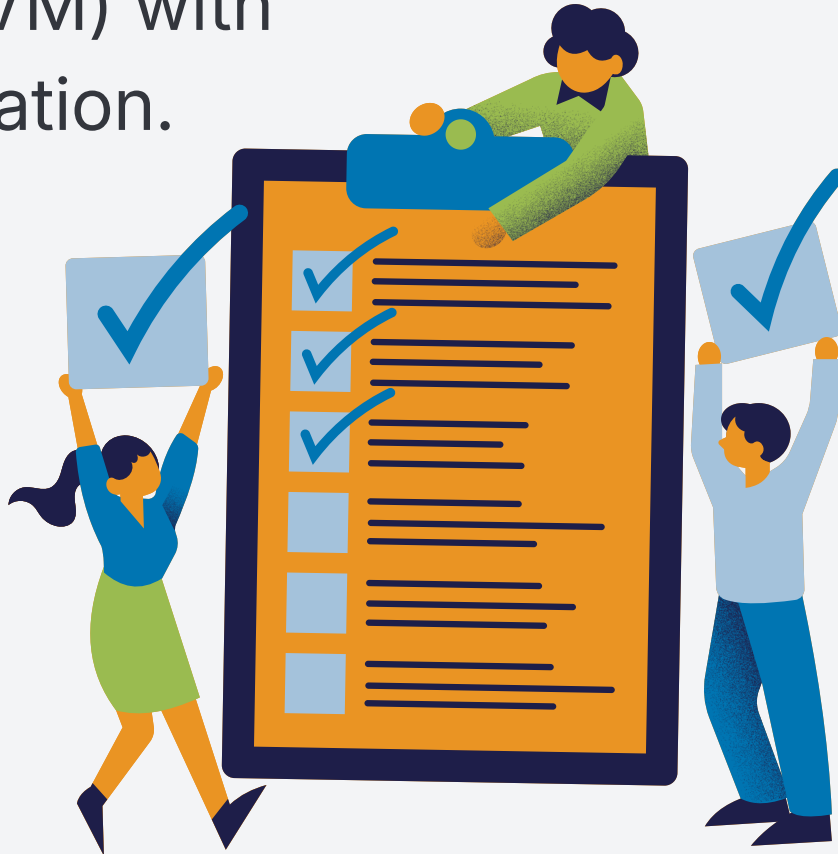
The research paper

Support Vector Machines with Hyperparameter Optimization Frameworks for Classifying Mobile Phone Prices in Multi-Class.



Main objective:

- Develop a more accurate mobile phone price classification model by combining Support Vector Machines (SVM) with Hyperopt and Optuna for hyperparameter optimization.
- Classify mobile phones into 4 price categories.
- Evaluate robustness using 5-fold cross-validation



Research peper

The reaserchpepers compared baseline SVM with SVM optimized using **Hyperopt** and **Optuna** under five-fold cross-validation.

Problem type Multiclass classification

Core model Support Vector Machie (SVM)

Optimization Hyperopt + Optuna



Hyperopt

Define an objective function
which to minimize -Accuracy

Define a configuration space of
hyperparameter using the hp
module

Set the fmin function, and
choose a search algorithm

Set the number of trials
(max_evals)

Run the optimization by
executing fmin function

Optuna

Define an objective function
which to maximize Accuracy

Define a configuration space of
hyperparameter using the
trial.suggest methods

Create a study object, and
choose a search algorithm

Set the number of trials
(n_trials)

Run the optimization by
executing study object

Motivation & reaserch GAP

Why this problem matters?

Consumer decisions

Estimate whether a phone's specifications are consistent with its price category.

Business strategy

Support pricing, product positioning, and competitive market analysis.

Market transparency

Help consumers and manufacturers better understand product value.



Data preprocessing and Steps reproducing

Yuna



Overall Workflow

1

Paper Review

2

Data Preprocessing

3

Data Split

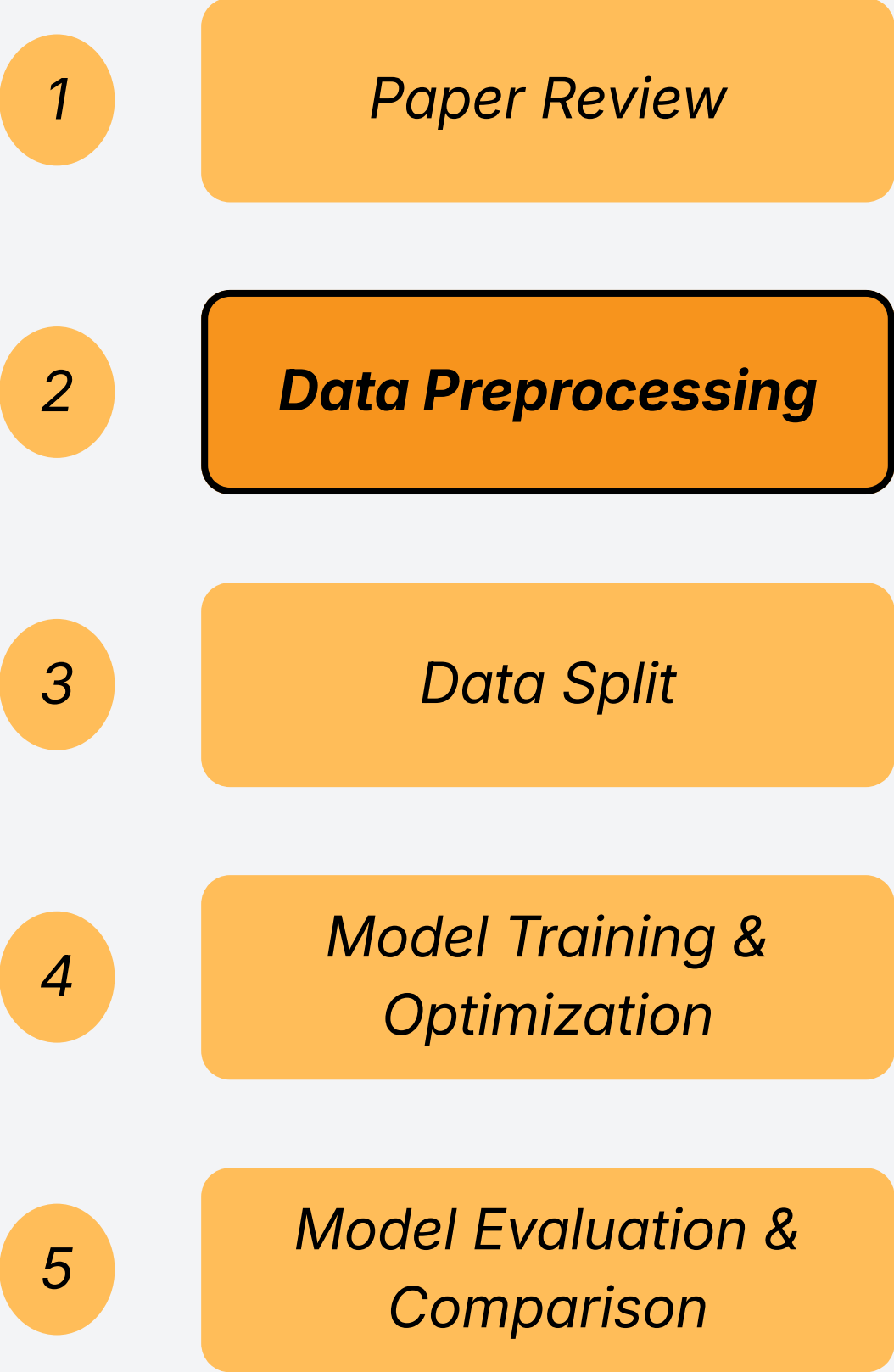
4

*Model Training &
Optimization*

5

*Model Evaluation &
Comparison*

Data Preprocessing



- Missing Values
- Outliers
- Duplicated Values
- Encoding & Scaling
- Check Target Values (Price_range)

Data columns (total 21 columns):				
#	Column	Non-Null Count		Dtype
0	battery_power	2000	non-null	int64
1	blue	2000	non-null	int64
2	clock_speed	2000	non-null	float64
3	dual_sim	2000	non-null	int64
4	fc	2000	non-null	int64
5	four_g	2000	non-null	int64
6	int_memory	2000	non-null	int64
7	m_dep	2000	non-null	float64
8	mobile_wt	2000	non-null	int64
9	n_cores	2000	non-null	int64
10	pc	2000	non-null	int64
11	px_height	2000	non-null	int64
12	px_width	2000	non-null	int64
13	ram	2000	non-null	int64
14	sc_h	2000	non-null	int64
15	sc_w	2000	non-null	int64
16	talk_time	2000	non-null	int64
17	three_g	2000	non-null	int64
18	touch_screen	2000	non-null	int64
19	wifi	2000	non-null	int64
20	price_range	2000	non-null	int64

Data Split

1

Paper Review

2

Data Preprocessing

3

Data Split



4

Model Training & Optimization

5

Model Evaluation & Comparison

- *Configure 5-Fold*
- *Split Dataset (Train : Test = 8 : 2)*
 - *Train dataset = 1600*
 - *Test dataset = 400*

5-Fold Cross Validation	All Data				
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Experiment 1	Testing data	Training data	Training data	Training data	Training data
Experiment 2	Training data	Testing data	Training data	Training data	Training data
Experiment 3	Training data	Training data	Testing data	Training data	Training data
Experiment 4	Training data	Training data	Training data	Testing data	Training data
Experiment 5	Training data	Training data	Training data	Training data	Testing data

Model Training & Optimization (1/2)

1

Paper Review

2

Data Preprocessing

3

Data Split

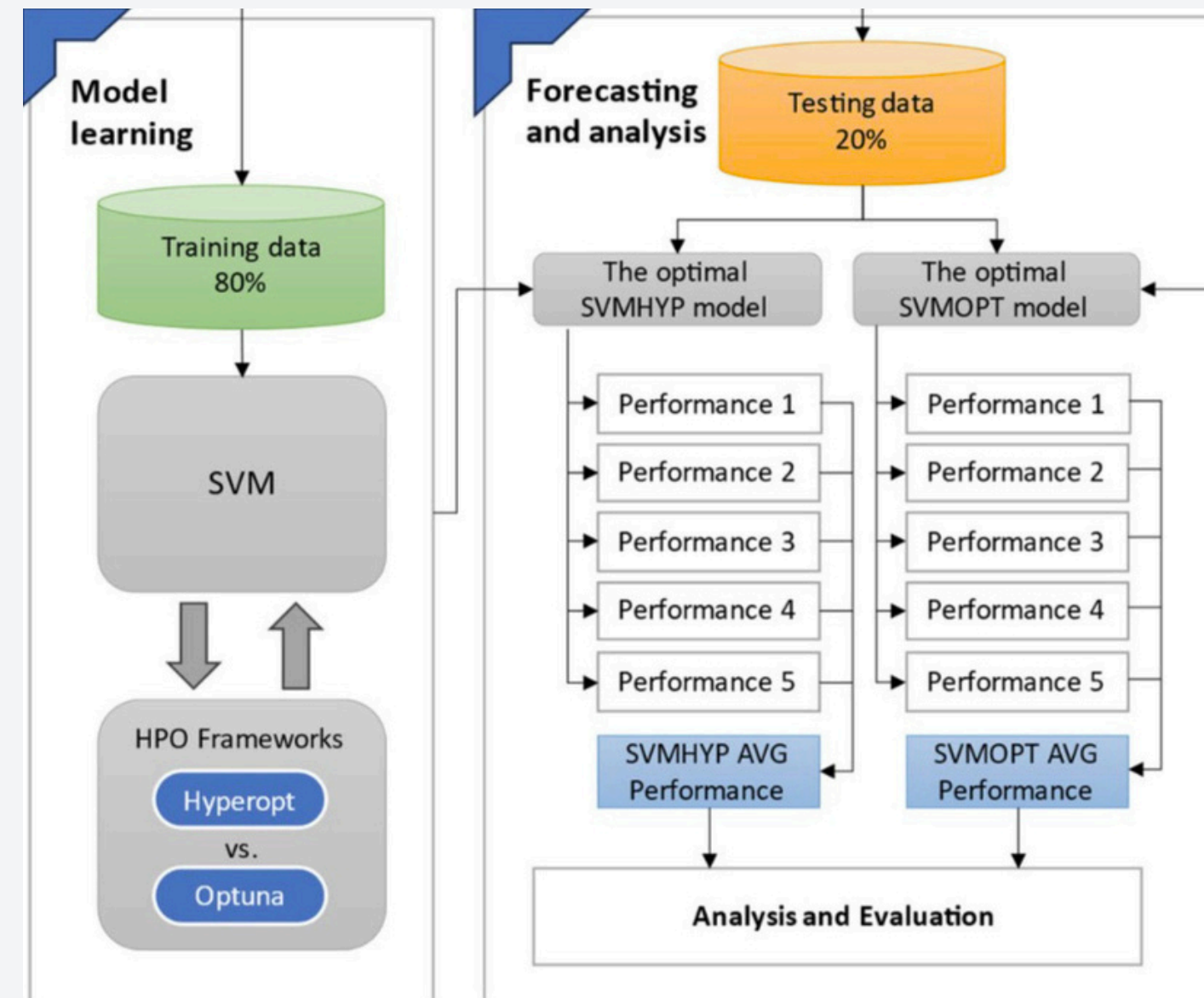
4

Model Training & Optimization

5

Model Evaluation & Comparison

- Baseline Model = SVM
- Use HPO Frameworks (Hyperopt vs Optuna) to find SVM hyper-parameters
 - *kernel, c, gamma, decision_function_shape*



```
space = {  
    "kernel": hp.choice(  
        "kernel",  
        ["linear", "poly", "rbf", "sigmoid"]  
    ),  
    "C": hp.uniform(  
        "C",  
        0.01,  
        100  
    ),  
}
```

```
"gamma": hp.uniform(  
    "gamma",  
    4.701e-7,  
    6.701e-7  
),  
"decision_function_shape": hp.choice(  
    "decision_function_shape",  
    ["ovo", "ovr"]  
),  
}
```


Model Training & Optimization (2/2)

1

Paper Review

2

Data Preprocessing

3

Data Split

4

Model Training & Optimization



5

Model Evaluation & Comparison

Hyperopt Results

	fold	accuracy	precision	recall	f1	kernel	C	gamma	decision_function_shape
0	1	0.9675	0.968579	0.9675	0.967406	poly	66.566724	5.121172e-07	ovo
1	2	0.9825	0.982714	0.9825	0.982497	linear	3.591615	4.938928e-07	ovr
2	3	0.9675	0.967667	0.9675	0.967523	poly	95.721973	6.240379e-07	ovo
3	4	0.9700	0.970797	0.9700	0.970219	linear	41.625730	5.503231e-07	ovo
4	5	0.9775	0.978013	0.9775	0.977337	linear	4.464916	5.973702e-07	ovr

display(optuna_df)

	fold	accuracy	precision	recall	f1	kernel	C	gamma	decision_function_shape
0	1	0.9725	0.972783	0.9725	0.972391	rbf	90.173688	5.098566e-07	ovr
1	2	0.9800	0.980196	0.9800	0.980049	poly	8.192525	6.374699e-07	ovr
2	3	0.9700	0.970396	0.9700	0.969967	poly	18.271770	5.628062e-07	ovr
3	4	0.9750	0.975248	0.9750	0.975049	poly	10.984054	5.439633e-07	ovo
4	5	0.9750	0.975681	0.9750	0.974793	linear	59.087318	4.979847e-07	ovr

Model Evaluation & Comparison (1/2)

1

Paper Review

2

Data Preprocessing

3

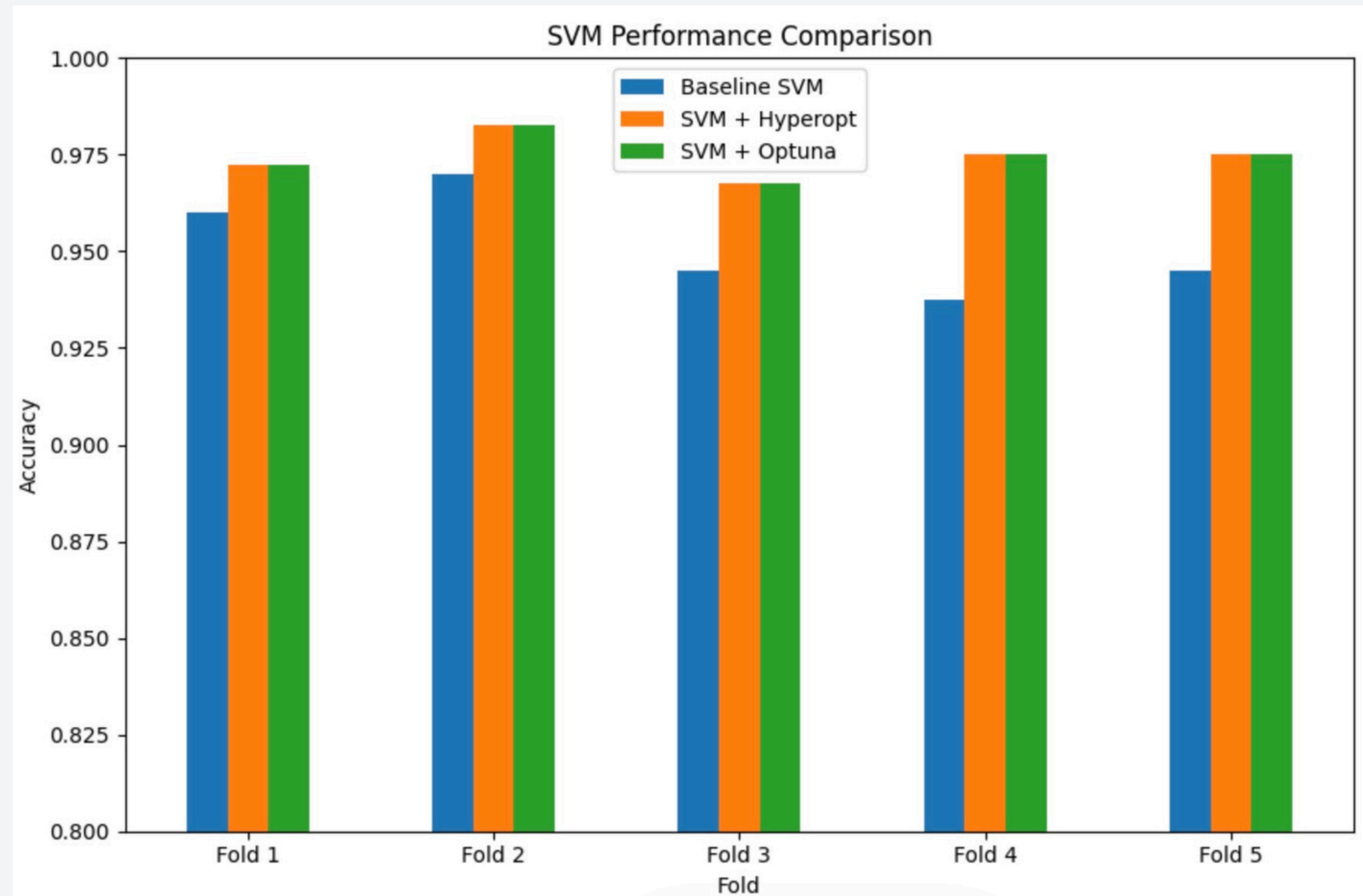
Data Split

4

*Model Training &
Optimization*

5

***Model Evaluation &
Comparison***



Model Evaluation & Comparison (2/2)

1

Paper Review

2

Data Preprocessing

3

Data Split

4

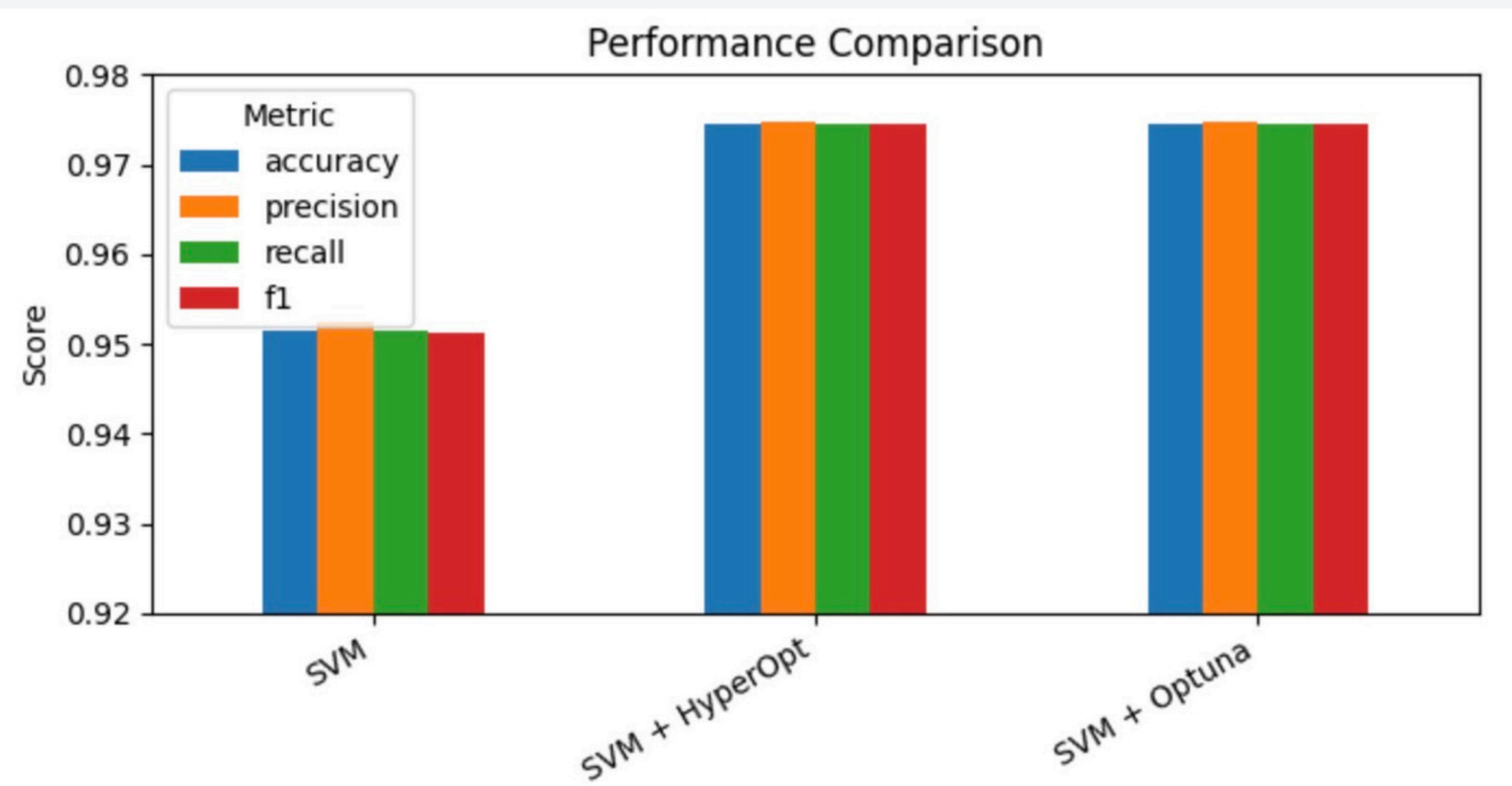
*Model Training &
Optimization*

5

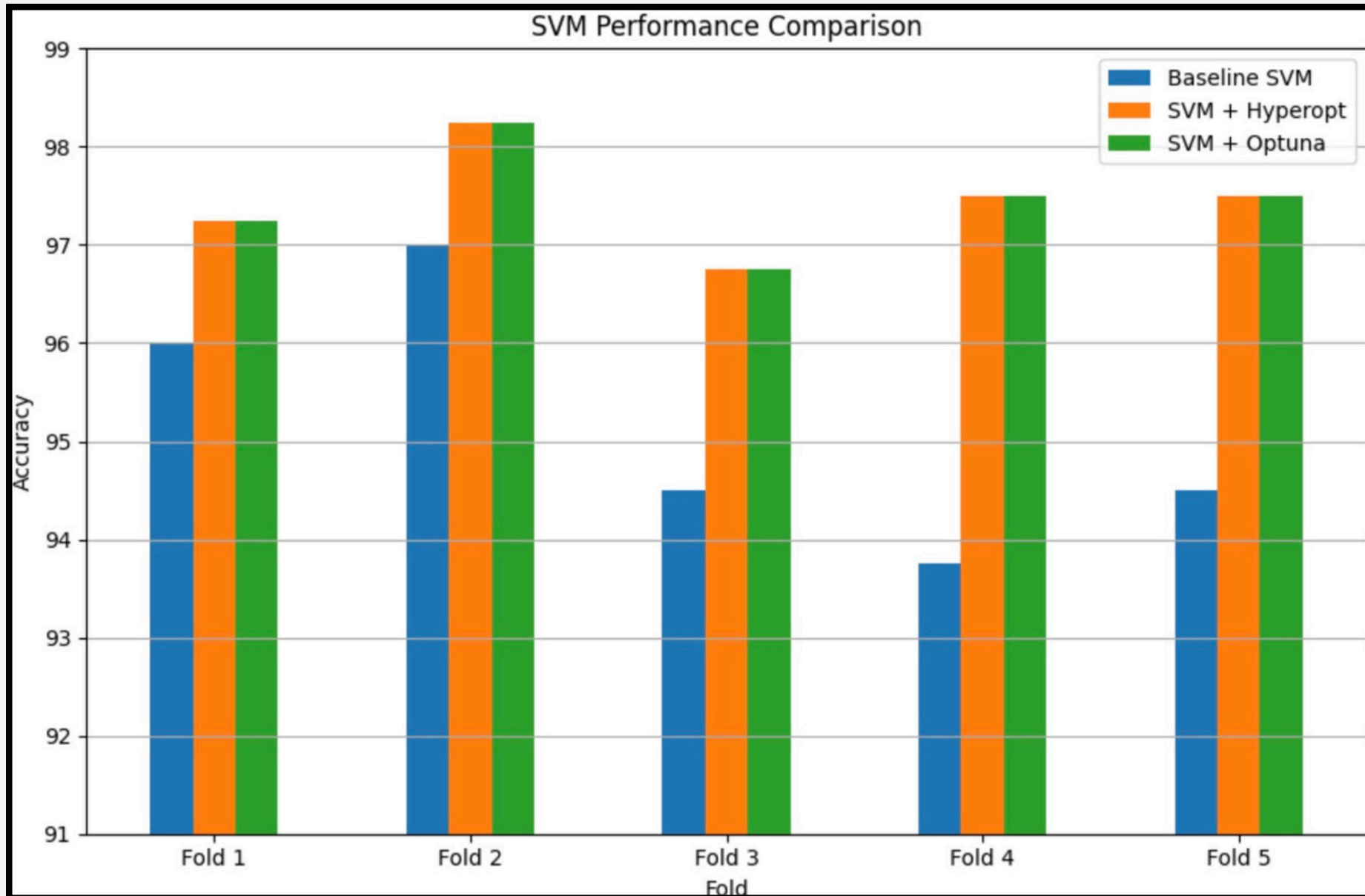
***Model Evaluation &
Comparison***



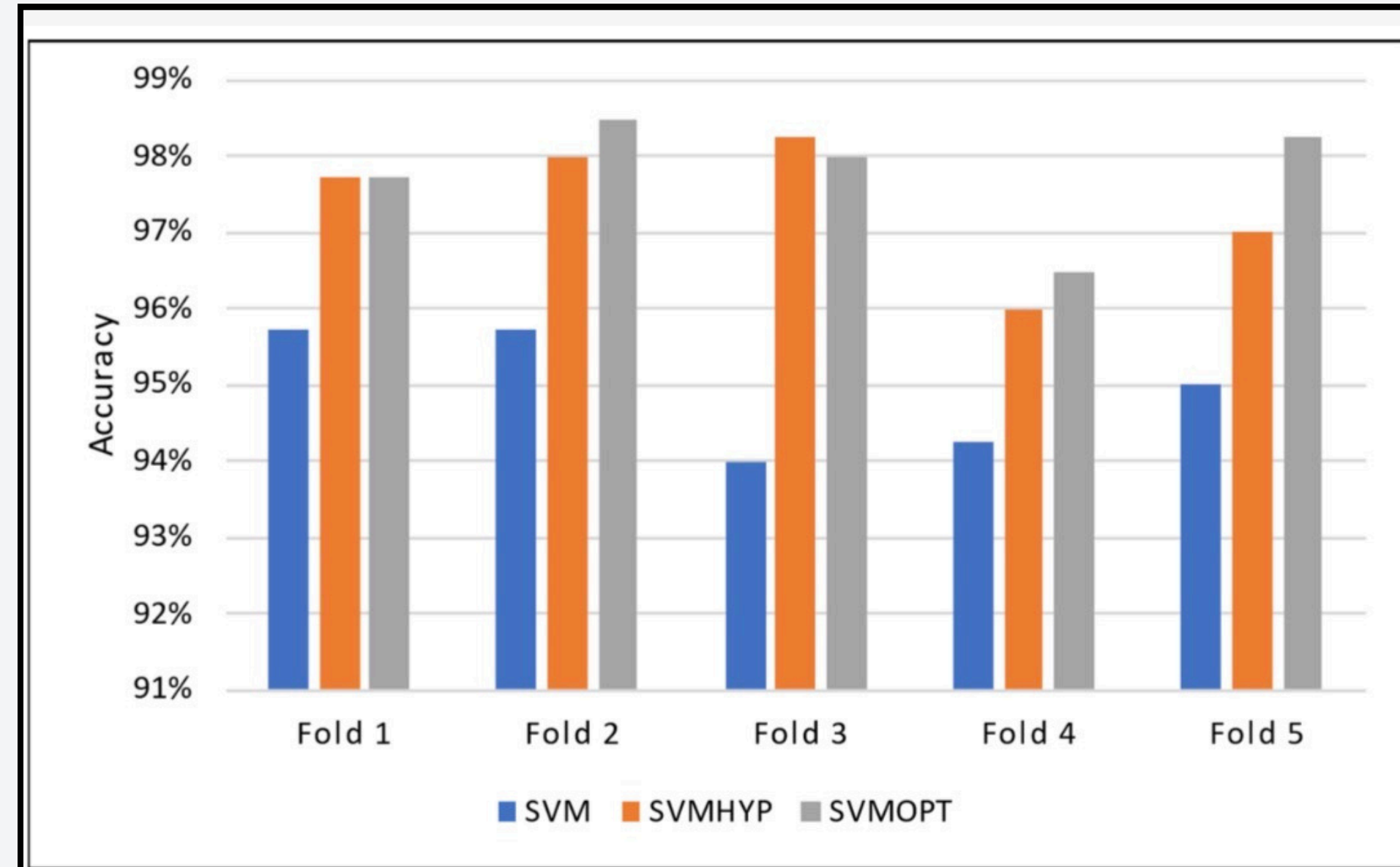
	accuracy	precision	recall	f1
SVM	0.9515	0.9524	0.9515	0.9512
SVM + HyperOpt	0.9745	0.9748	0.9745	0.9744
SVM + Optuna	0.9745	0.9748	0.9745	0.9745



Comparison with Research Paper



*Project Result
(Accuracy)*



*Research Paper Result
(Accuracy)*

Contribution and Improvement

Cotton





Research Replication and Project Contribution

Reproduce the original research methodology using the mobile price

Establishing a comparable baseline and improving the hyperparameter-validation process

Testing Logistic Regression as an additional machine learning model

The baseline replication was very close to the paper





Improvement: Hyperopt and Optuna HPO

Hyperparameters optimized:

- Kernel: Linear, Polynomial, RBF, Sigmoid
- Regularization parameter C
- gamma
- Multiclass decision strategy



Improved validation approach:

- 100 optimization trials per outer fold
- Hyperopt uses TPE
- Optuna uses TPE Sampler
- Inner Stratified CV selects hyperparameters
- Outer fold remains unseen until final evaluation
- Reduces risk of data leakage and overfitting



Improvement: Hyperopt and Optuna HPO

Model	Accuracy	Macro F1
Baseline SVM	95.15%	95.12%
SVM + Hyperopt	97.60%	97.60%
SVM + Optuna	97.60%	97.60%





Contribution:

Logistic Regression

Why add Logistic Regression?

- Provides a simpler alternative to SVM
- Supports multiclass classification
- More interpretable than SVM
- Useful for checking whether a complex model is really necessary

Workflow improvement:

- StandardScaler + Logistic Regression
- Combined using a Scikit-learn Pipeline
- Scaling learned only from each training fold
- Prevents preprocessing leakage
- Same 5-fold Stratified CV for fair comparison





Contribution:

Logistic Regression

Model	Accuracy
Baseline SVM	95.15%
Logistic Regression	~96.10%
SVM + Hyperopt	97.60%
SVM + Optuna	97.60%



Overall Contribution and Conclusion

Main contributions

- Successfully reproduced the original SVM results
- Added a more reliable inner/outer CV structure for HPO
- Compared Hyperopt vs Optuna under the same conditions
- Added Logistic Regression as an alternative classifier
- Used a leakage-safe StandardScaler Pipeline for Logistic Regression
- Demonstrated that preprocessing must be chosen based on the model

Baseline SVM

95.15%

Logistic Regression

96.10%

→ +0.95 percentage points

SVM + Hyperopt/ Optuna

97.60%

→ +2.45 percentage points



Conclusion and Challenges

Kai



Interpretation of Results



- Hyperparameter optimization improved SVM performance
 - Baseline SVM: 95.15%
 - Hyperopt SVM: 97.60%
 - Optuna SVM: 97.60%
- Hyperopt and Optuna achieved the same best accuracy
 - +2.45 percentage points compared with the baseline



Interpretation of Results

- Logistic Regression achieved 96.10%
 - Stronger than the baseline SVM
 - Still lower than the optimized SVM models
- Overall: Hyperparameter optimization was effective for improving mobile phone price classification performance.



Challenges

Difficult to understand the research paper and figure out the reproducing steps.

Difficult to apply theory to real-world situations.

Difficult to compare and interpret the performance of different machine learning models.

Complexity of capturing nonlinear relationships between mobile-phone specifications and price classes

Understanding and implementing Hyperopt and Optuna, both tools are used for hyperparameter optimization, but they work differently.

Thank You

Questions are welcome

